

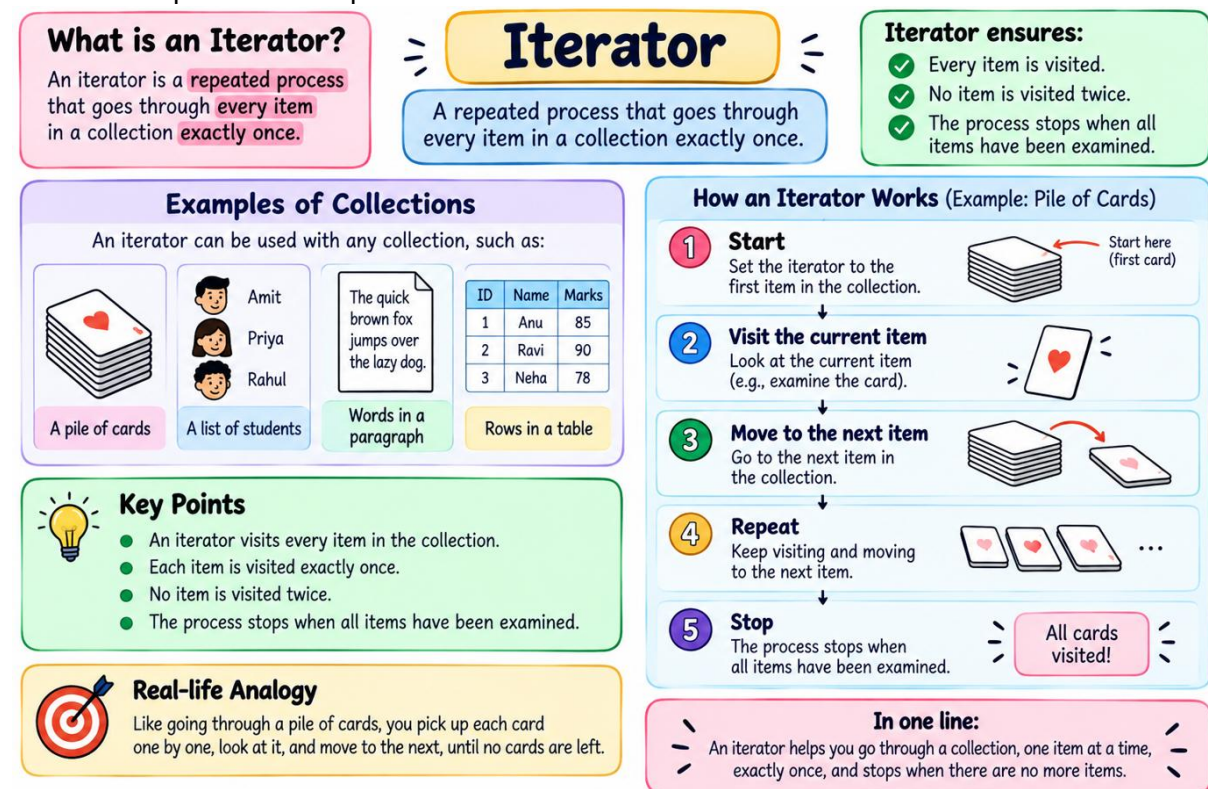
What is an Iterator?

An **iterator** is a repeated process that goes through every item in a collection exactly once.

Examples of collections: a pile of cards, a list of students, words in a paragraph, rows in a table.

The iterator ensures:

- Every item is visited.
- No item is visited twice.
- The process stops when all items have been examined.



Why Use an Iterator?

Computers are good at repetition. Humans get bored and make mistakes when repeating the tasks.

An iterator gives a clear, repeatable pattern for:

- Counting items.
- Adding values from each item.
- Checking each item for some condition.
- Collecting information from each item.

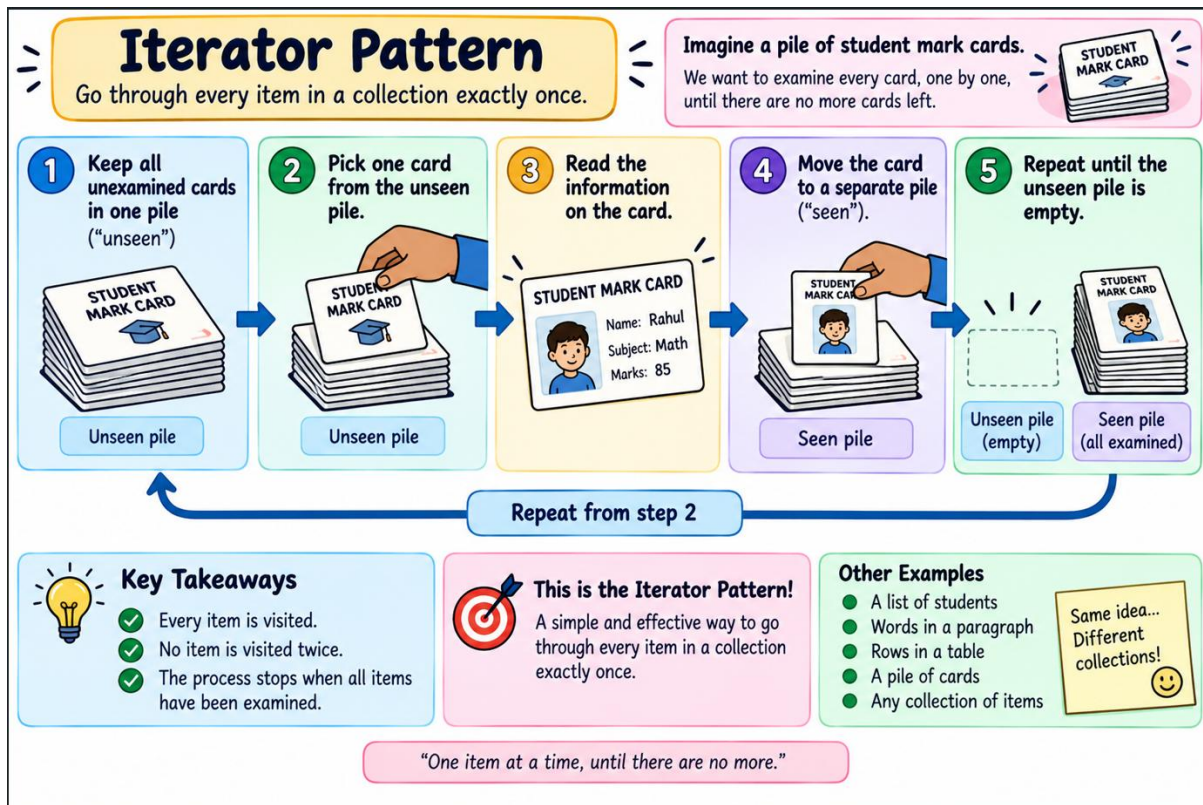
Everyday Analogy

Imagine a pile of student mark cards.

To examine every card:

1. Keep all unexamined cards in one pile ("unseen").
2. Pick one card from the unseen pile.
3. Read the information on the card.
4. Move the card to a separate pile ("seen").
5. Repeat until the unseen pile is empty.

This is the iterator pattern.



Parts of an Iterator

1. Initialisation

Prepare everything before repetition begins.

Examples:

- Arrange all cards in an "unseen" pile.
- Keep space ready for a "seen" pile.
- Set any counting or sum variables to zero (if needed).

2. Check Whether to Continue

Ask: Are there still items in the unseen pile?

- If **yes**, continue.
- If **no**, stop.

This check happens **before** picking the next item. This also handles the case where the pile is empty from the start.

3. Pick and Examine One Item

Take one item from the unseen pile.

- For an unordered pile, any card can be picked.
- For an ordered collection (like words in a sentence), always pick the top/first item to keep the order.

Read or process the information on the item.

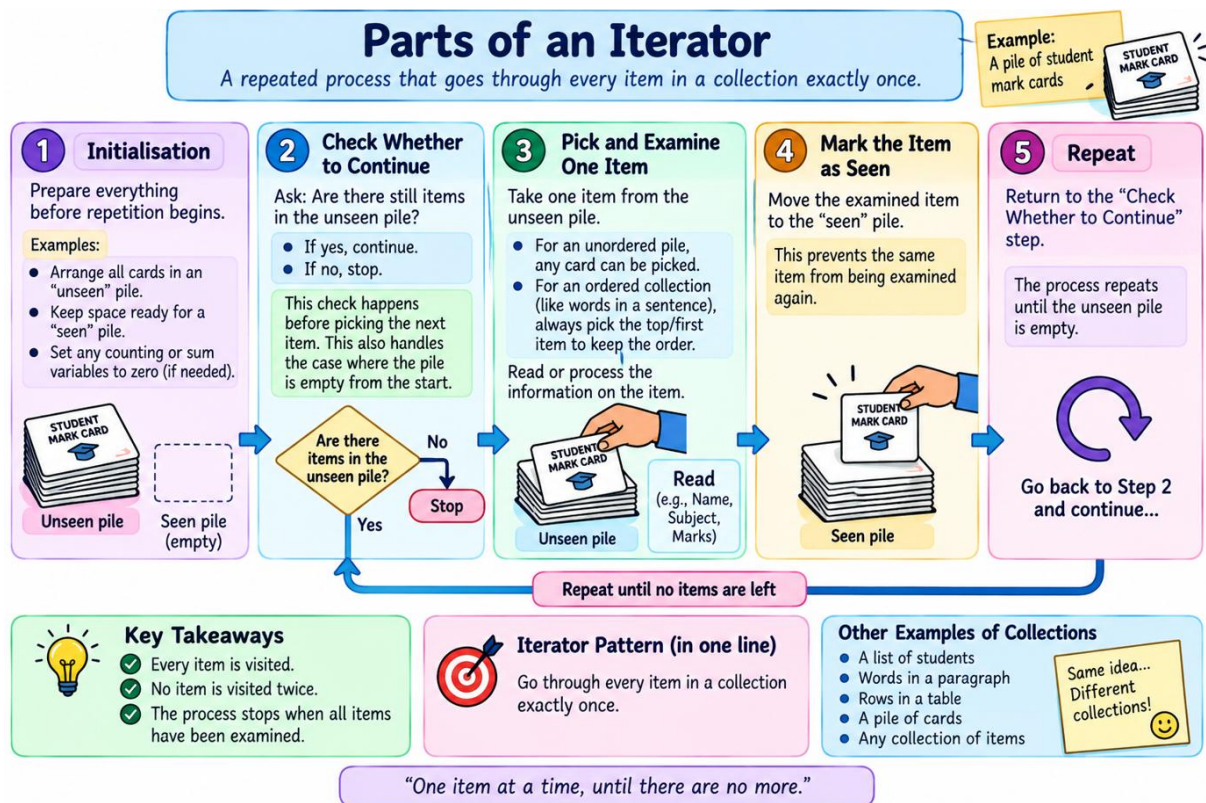
4. Mark the Item as Seen

Move the examined item to the "seen" pile.

This prevents the same item from being examined again.

5. Repeat

Return to the "Check Whether to Continue" step.



Generic Iterator Steps

text

Step 1: Initialisation – arrange all items in an “unseen” pile

Step 2: If no items remain in “unseen”, exit; otherwise continue

Step 3: Pick one item from “unseen”, examine it, move it to “seen”

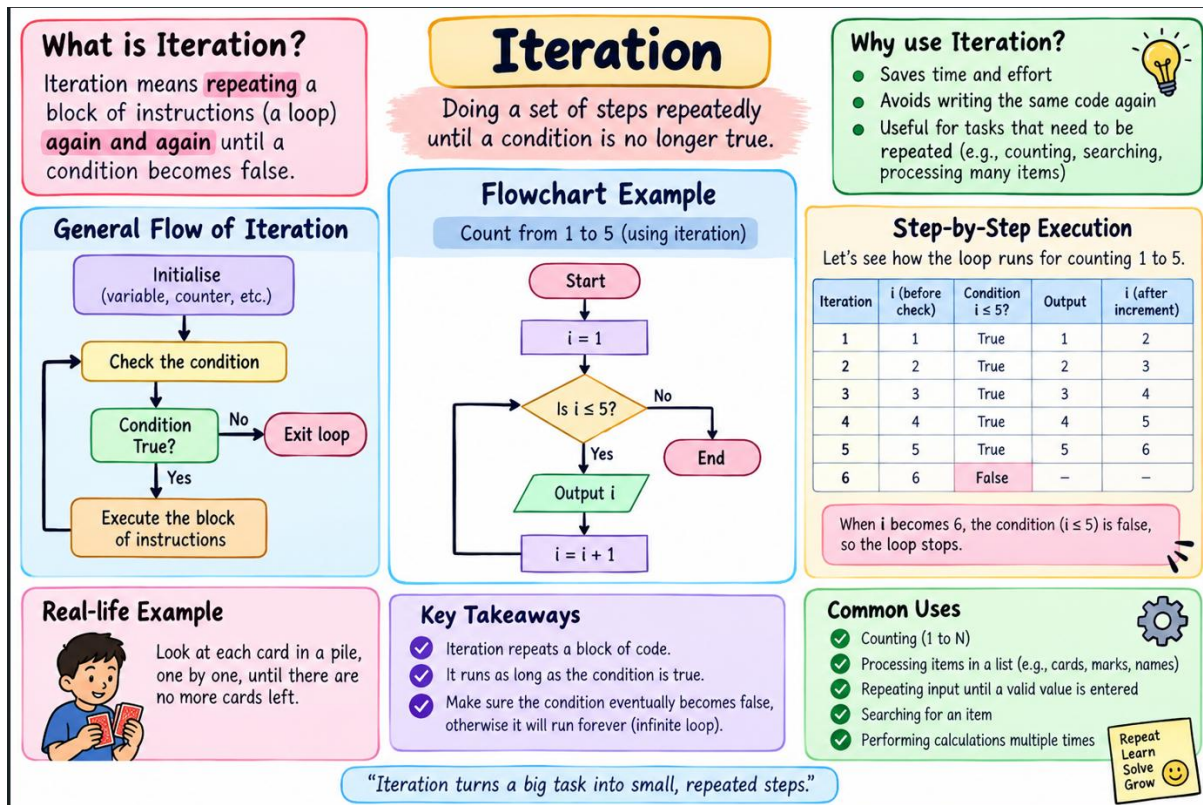
Step 4: Go back to Step 2

Visualising with a Flowchart

A **flowchart** is a diagram that shows the steps of a process.

Four key symbols:

- **Rounded box:** Start or End
- **Rectangle:** An action or step
- **Diamond:** A decision (Yes/No or True/False)
- **Arrow:** Shows the direction of flow



Generic iterator flowchart:

```

text
Start
↓
Initialisation
↓
More items in "unseen"? -No→ End
↓Yes
Pick and examine one item; move to "seen"
↓
(return to decision)
    
```

Iterator for Cards (Specific Example)

```

text
Start
↓
Initialisation:
- Arrange cards in "unseen" pile
- Keep free space for "seen" pile
↓
More cards in "unseen" pile? -No→ End
↓Yes
Pick one card from "unseen"
Examine the card
Move card to "seen" pile
↓
(return to decision)
    
```

Important Ideas

- An iterator is not just “doing something again.” It is a **systematic** way to visit each item once.
- The **two-pile method** (unseen and seen) guarantees:
 - No item is missed.
 - No item is repeated.
- The stopping condition is clear: stop when the unseen pile is empty.

Connection to Later Topics

- **Variables:** To store counts, sums, or other information while iterating.
- **Filtering:** To process only some items that meet a condition.
- **Nested iterations:** To compare each item with every other item.

Complexity Note (High-Level Only)

The structure of an iterator tells us how much work is involved:

- Iterating over all items: order N , written as **$O(N)$** .
- Iterating over all pairs of items: order N^2 , written as **$O(N^2)$** .

This idea becomes important when working with large datasets.

These notes cover the complete iterator chapter: definition, purpose, steps, flowchart representation, and the card-based example, using only the textbook content in simple, beginner-friendly language.
flowcharts

Flowcharts – Teaching Notes (Chapter 2.2)

What is a Flowchart?

A **flowchart** is a diagram that shows the steps of a process in order.

It helps us:

- See the flow of a procedure clearly.
- Spot repeating patterns.
- Explain a method to another person.
- Reuse a solution for similar problems.

Why Use Flowcharts?

Our eyes are good at recognising visual patterns. A flowchart turns a written procedure into a picture, making it easier to:

- Understand the sequence of steps.
- Identify where decisions are made.
- See where repetition (loops) happens.

Four Key Symbols

Only four symbols are needed for the flowcharts in this book:

1. Terminal Symbol (Rounded Box)

- Shows the **Start** or **End** of the flowchart.
- Every flowchart begins and ends with this symbol.

2. Process Symbol (Rectangle)

- Represents an **action** or **activity**.
- Examples: “Arrange cards in unseen pile”, “Increment count”, “Add $X.F$ to sum”.

3. Decision Symbol (Diamond)

- Represents a **condition check**.
- The condition is either **True (Yes)** or **False (No)**.
- Two branches come out of the diamond, one for each outcome.

4. Arrow

- Shows the **direction of flow** from one step to the next.
- Connects all the symbols in order.

Generic Iterator Flowchart

The iterator pattern can be drawn as a flowchart:

text

[Start]

↓

[Initialisation step]

↓

<Diamond: Continue iteration?>

↓No→ [End]

↓Yes

[Step to be repeated]

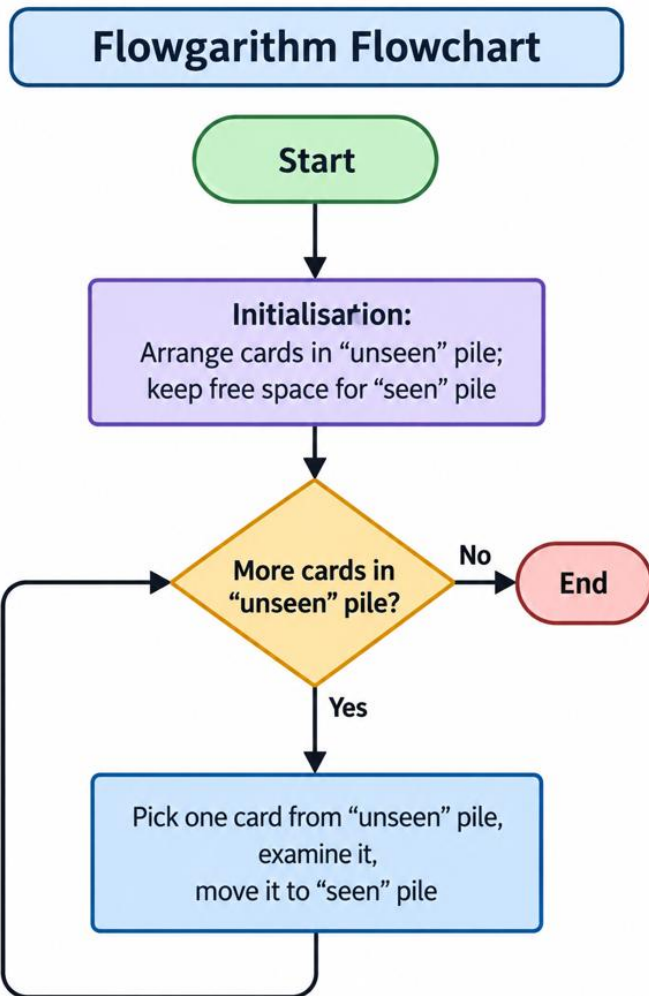
↓

(return to decision diamond)

Key points:

- The **decision** comes before the repeated step. This handles the case where there are no items to begin with.
- The **loop** is visible: from the decision diamond, through the repeated step, and back to the decision.
- Any process with such a loop can be modelled using an iterator.

Iterator Flowchart for Cards



How to Read a Flowchart

1. Start at the **Start** terminal.
2. Follow the arrows in order.
3. At a **diamond**, check the condition:
 - If **True**, follow the "Yes" branch.
 - If **False**, follow the "No" branch.
4. Stop when you reach the **End** terminal.